MA-ROPE: Multi-RAT Privacy-Preserving Federated Deep Reinforcement Learning for V2X Task Offloading

Anonymous Authors

***Abstract*—Autonomous vehicles increasingly run computation-heavy tasks with strict service-quality demands that their on-board processors cannot always meet on their own, which is why offloading part of that work to the network edge has become common practice. When many vehicles compete for the same wireless link and the same edge server, however, a policy that offloads aggressively without regard for channel conditions or queue congestion can fail badly, and this failure mode is easy to miss if a system is only ever tested at one vehicle density. This paper presents MA-ROPE, a Multi-RAT (Radio Access Technology) Privacy-Preserving Federated Deep Reinforcement Learning framework for Vehicle-to-Everything (V2X) task offloading. MA-ROPE jointly chooses among four radio interfaces (LTE, ITS-G5, Wi-Fi 6, and 5G-NR) and decides how much of each task to offload, using a Dueling Double Deep Q-Network (D3QN) whose training is coordinated across vehicles through Federated Averaging (FedAvg) so that raw sensor data never leaves the vehicle. Under dense, latency-critical traffic, MA-ROPE significantly improves the rate at which tasks are completed on time compared to fixed single-radio baselines. We also introduce the Deadline Retention Ratio, a new metric for how much of a policy's on-time performance survives a large increase in fleet size, and find that MA-ROPE holds onto nearly all of its performance while baseline methods experience severe degradation. Furthermore, the joint radio-selection and offloading logic adds no measurable computational overhead over simpler controllers. Ultimately, radio selection, partial offloading, and edge-queue contention must be optimized jointly, not separately, for vehicular networks to deliver time-critical tasks reliably as they scale.**

***Index Terms*—Task offloading, Vehicle-to-Everything (V2X) communication, vehicular edge computing, federated deep reinforcement learning, multi-RAT selection, heterogeneous networks, edge computing, autonomous driving, privacy-preserving learning, Jakes fading.**

I. Introduction

Vehicle-to-Everything (V2X) communication enables safety-critical and compute-intensive applications such as cooperative perception, autonomous driving, and real-time hazard avoidance. These applications impose heterogeneous Quality-of-Service (QoS) requirements: safety messages may become unusable within approximately 100 ms, whereas compute and infotainment workloads can tolerate substantially larger delays. At the same time, processing the growing volume of camera, radar, and LiDAR data entirely on resource-constrained vehicle Electronic Control Units (ECUs) is increasingly difficult. Vehicular Edge Computing (VEC) addresses this limitation by offloading computation to roadside Multi-access Edge Computing (MEC) servers.

The effectiveness of offloading, however, depends jointly on where computation is performed and how the offloaded workload reaches the edge. No single Radio Access Technology (RAT) is uniformly optimal across task types and mobility conditions. Long-Term Evolution (LTE), ITS-G5 (the European Intelligent Transport Systems band at 5.9 GHz), WiFi-6, and 5G New Radio (5G-NR) exhibit different latency, throughput, coverage, and mobility characteristics. A RAT suitable for a delay-tolerant compute task may therefore be unsuitable for a safety-critical task generated by the same vehicle. Moreover, binary local/offload decisions cannot exploit parallel local and MEC processing. These observations motivate treating RAT selection and partial computation offloading as a joint decision rather than as independent optimization stages.

A second challenge emerges as vehicle density increases. Vehicles associated with the same Roadside Unit (RSU) compete not only for wireless resources but also for a shared MEC processing queue. Consequently, aggressive offloading that performs well for an isolated vehicle may degrade when many vehicles simultaneously submit workloads. This degradation can also be hidden by conventional communication metrics. Packet Reception Ratio (PRR) counts a packet as successfully delivered even when it arrives after its application deadline; for safety-critical traffic, however, a late result may be operationally equivalent to a lost one. Evaluation must therefore distinguish successful reception from deadline-compliant delivery.

To address these coupled challenges, we propose MA-ROPE, a multi-agent framework that jointly learns RAT selection and partial computation offloading under shared MEC contention. The core insight is that RAT choice and offload ratio are mutually dependent: the selected RAT determines transmission latency and reliability, which in turn determines the feasible offload ratio; conversely, the offload ratio sets transmission duration, which affects queue contention and RSU delay for all co-located vehicles. Optimizing these decisions sequentially yields suboptimal policies because downstream interactions are ignored.

Each vehicle selects among LTE, ITS-G5, WiFi-6, and 5G-NR while adapting its offloaded workload fraction $\rho \in \{0.25, 0.50, 0.75, 1.0\}$ to channel conditions, task requirements, energy budget, and current RSU queue state. The MDP state explicitly includes shared queue depth Q, enabling each vehicle's policy to account for the decisions of co-

Submitted for anonymous peer review

located vehicles. A Dueling Double Deep Q-Network (D3QN) decomposes value and advantage streams, enabling stable learning under multi-vehicle interactions. Federated Averaging (FedAvg) coordinates policies across vehicle clusters without requiring vehicles to share raw observations, preserving operational privacy while enabling cooperative learning at the policy level.

Accordingly, this work investigates whether a federated multi-RAT offloading policy can preserve deadline-compliant V2X delivery as fleet density and shared MEC contention increase.

Despite extensive research on VEC offloading, existing approaches generally address only subsets of heterogeneous RAT selection, partial computation offloading, multi-vehicle MEC contention, mobility-aware channel behavior, and federated cooperative learning. Recent distributed deep RL methods (e.g., [**?**], [1]) employ federated learning or game-theoretic coordination, but typically optimize RAT selection or offload ratio independently, rather than jointly. Bandit-based approaches (e.g., [2], [**?**]) adapt network selection to density, but do not model partial execution or shared queue dynamics. Task-splitting strategies (e.g., [3]) optimize fairness or latency, but rely on analytical optimization rather than learning from channel variability and queue state. No prior work jointly optimizes RAT selection, partial-offload ratio, and shared RSU queue contention within a federated deep RL framework that explicitly models deadline compliance across multiple fleet densities.

MA-ROPE addresses this gap by bringing these dimensions together and introducing a new evaluation discipline: not only performance at a fixed operating point, but critically, how well deadline-compliant service survives increasing fleet contention. We quantify this via the Deadline Retention Ratio (DRR), which measures how much of a policy's reference-density on-time performance remains at higher densities. This metric reveals that aggressive offloading policies that perform well at low vehicle density can collapse catastrophically as the fleet grows, a failure mode invisible to scalar operating-point metrics.

The main contributions of this work are:

First, we formulate RAT selection and partial computation offloading as a unified MDP decision, not as sequential stages. Unlike prior work that selects the RAT first and then optimizes offload ratio (or vice versa), we model the action space as (RAT, ρ) pairs, directly encoding the mutual dependence. Critically, the MDP state includes the shared RSU queue depth, enabling each vehicle to account for others' decisions and avoid the "AllOffload collapse" observed when vehicles naively offload all traffic without considering queue saturation. Second, We coordinate D3QN training across vehicle clusters via Federated Averaging (FedAvg), enabling cooperative learning without centrally pooling raw vehicle observations. The key innovation is that the per-agent state explicitly includes the shared RSU queue depth, making each vehicle's learned policy aware of co-located vehicle demand. This differs from pure distributed RL by incorporating shared resource constraints at the agent level, not requiring a central coordinator during execution. Third, We distinguish communication reliability (PRR) from application-useful on-time delivery (OTR). We introduce the DRR, which normalizes a policy's on-time delivery at a target density relative to its own reference-density performance, enabling fair comparison across heterogeneous policies without dependence on utility-function constants. Finally, We evaluate MA-ROPE and four baselines across multiple fleet sizes ($N \in \{8, 16, 32\}$) under URLLC traffic. We demonstrate that while baseline policies suffer catastrophic failure modes at high contention, the proposed federated MDP formulation robustly maintains deadline compliance, link reliability, and low computational overhead across varying densities.

II. Related Work

A. Task Offloading in Vehicular Networks

Existing V2X offloading approaches broadly include heuristic, optimization-based, and learning-based methods. Greedy and heuristic schemes select offloading destinations using predefined decision rules, but their ability to adapt to rapidly varying channel conditions and shared-resource contention is limited. Bandit and tabular approaches such as UCB [2] introduce online adaptation but are better suited to relatively small and stationary decision spaces. Control-theoretic methods based on Lyapunov optimization [3] explicitly address queue stability, but generally optimize time-averaged objectives rather than task-specific deadline compliance. Deep Reinforcement Learning (DRL) approaches [3]–[5] provide greater adaptability, although many existing formulations simplify either the communication interface or the computation decision through single-RAT operation or binary local/offload actions.

Guo et al. [1] proposed DTOF/HIOA, combining a Double DQN with a Long Short-Term Memory (LSTM) network and Nash-equilibrium-based optimization in a three-tier cloud-edge-vehicle architecture over 5G-NR. Although the framework achieves strong delay performance, it does not jointly consider heterogeneous RAT selection, federated cooperation, and shared RSU queue contention across co-located vehicles.

A complementary line of work considers partial offloading, where a task is divided between local and MEC execution rather than being processed entirely at one location. Parallel local and remote execution can reduce completion time by exploiting both computing resources simultaneously. Prior work has established such parallel-execution models [6], including formulations in which total completion delay is governed by the slower of the local and remote processing branches. In most such approaches, however, the offload ratio is optimized independently of heterogeneous radio selection. MA-ROPE instead couples these decisions: the amount of computation transmitted and the RAT used for transmission are jointly determined, with the realized offload ratio adapting to the SINR of the selected interface.

B. Multi-RAT and Heterogeneous Access Selection

Heterogeneous V2X access has traditionally been studied primarily as a communication-selection problem. Yacheur et al. [7] developed Double DQN based selection between ITS-G5 and LTE-V2X, demonstrating the benefit of adapting RAT

choice to channel conditions. Their physical-layer parameters and SINR-based Division Mode motivate elements of the heterogeneous channel and partial-offloading models adopted in MA-ROPE.

More recent work has expanded heterogeneous access beyond two interfaces, with studies considering multi-RAT assignment and heterogeneous V2X resource allocation across combinations such as Dedicated Short-Range Communications (DSRC), cellular V2X, and millimeter-wave (mmWave) links. Mirza et al. [8] proposed the Mobility, Contact, and computational Load-Aware (MCLA) task offloading framework for heterogeneous 5G-NR-V2X edge networks, incorporating mobility, contact duration, MEC load awareness, and task partitioning. Their scheme supports load-aware server selection and partial execution, but uses model-based decision logic rather than a learned federated joint RAT-offload policy.

These studies establish the value of heterogeneous access, but their decision structures remain different from the problem addressed here. RAT-selection methods generally determine the interface over which an already-defined workload is transmitted, whereas computation offloading must additionally determine how much workload should be transmitted. The two decisions interact directly because the selected RAT affects transmission latency and reliability, while the offloaded fraction determines transmission duration, energy consumption, local computation demand, and MEC load.

MA-ROPE therefore represents RAT identity and workload partitioning within a common decision process rather than optimizing them as separate stages.

C. Federated and Privacy-Preserving Learning for VEC

Cooperative learning has increasingly been applied to vehicular edge resource management. Centralized multi-agent approaches such as Multi-Agent Deep Deterministic Policy Gradient (MADDPG) [5] can exploit joint agent information during training and provide strong coordination, but require centralized access to information that remains local under a federated formulation.

Federated learning offers an alternative by aggregating model parameters rather than centrally collecting each vehicle's training observations. Moon and Lim [9] proposed federated Deep Deterministic Policy Gradient (DDPG) learning for vehicular edge task offloading and power control, combining local actor-critic learning with periodic model aggregation. Their formulation incorporates queue information and power control, but does not perform heterogeneous RAT selection or explicit partial-offload-ratio selection. Other recent federated VEC approaches employ Double Deep Q-Network (DDQN) methods or multi-agent DRL with FedAvg for collaborative offloading, but similarly focus on destination or mode selection rather than a joint RAT-partial-offload action.

Accordingly, the contribution of MA-ROPE is not simply the combination of federated learning and multi-RAT access. Rather, FedAvg is applied to a policy whose decision space jointly captures heterogeneous RAT selection, workload partitioning, and queue-aware offloading.

The privacy benefit should also be interpreted carefully. FedAvg avoids centralized pooling of raw observations, but exchanging model updates does not itself provide formal privacy guarantees because trained parameters may remain vulnerable to inference attacks [10]. Secure aggregation and differential privacy are therefore complementary mechanisms for production deployment.

D. Research Gap and Positioning

The literature demonstrates substantial progress in each component of vehicular edge offloading, but these components are predominantly addressed in isolation. We position MA-ROPE against the most closely related recent works across several dimensions. Federated multi-agent DRL for VEC has been proposed for task offloading, combining local actor-critic learning with periodic model aggregation [**?**]. However, such formulations often do not perform heterogeneous RAT selection or model explicit partial-offload-ratio choice. Similarly, combining federated DDPG with queue information [9] lacks optimization over heterogeneous RATs or joint RAT-ρ decisions.

Multi-RAT offloading under distributed learning has also been explored, such as 5G URLLC/eMBB with distributed DRL [4] or cloud-edge-vehicle offloading with game theory [1]. While these address RAT diversity, they typically optimize RAT choice and offload destination separately, without jointly learning ρ. Additionally, they evaluate at a single or limited set of fleet densities and do not quantify how performance scales.

Density-aware network selection using bandit-based and heuristic approaches [**?**], [2] adapts network selection to vehicle density, but operates in a lower-complexity decision space and typically assumes fixed offload destinations. Task splitting and fairness constraints have been modeled by optimization-based approaches [3], but these rely on analytical solutions rather than learning from channel variability, and do not address the adaptive multi-RAT selection problem.

MA-ROPE differs from each of these in that it (1) jointly optimizes RAT selection and partial offload ratio within a unified MDP, (2) explicitly models shared RSU queue contention in the agent state to avoid naive all-offload policies, and (3) introduces and evaluates the Deadline Retention Ratio across multiple fleet densities—a metric absent from prior work. Table I summarizes the positioning of MA-ROPE relative to representative prior approaches.

TABLE I
LITERATURE COMPARISON: FEATURE COVERAGE. FL: FEDERATED LEARNING.

Paper	Multi-RAT	Partial ρ	Queue Cont.	FL Privacy	Jakes	Yacheur Ch.
Guo 2026 [1]	×	Partial	Per-node	×	×	×
Yacheur 2023 [7]	✓ITS+LTE	×	×	×	✓	✓
Wu 2021 [2]	×	×	×	×	×	×
MA-ROPE (ours)	**✓4 RATs**	**✓$\rho \in \{0..1\}$**	**✓shared RSU**	**✓FedAvg**	**✓Jakes**	**✓adopted**

E. Contribution Decomposition

Fig. 2 decomposes MA-ROPE's utility into per-component contributions at the reference fleet size of N=8. This ablation predates the local-latency and Jakes-fading corrections described in Section V and has not been rerun under the



Fig. 1. Literature gap matrix. MA-ROPE (bottom row, blue highlight) is the only framework with all six properties: supported, partial, or not supported.

corrected physics; each of its five stages (AllOffload, adding shared-queue awareness, multi-RAT selection, the DRL policy, and FedAvg) would require training a separate intermediate model variant to reproduce, which is left for future work. It is reported here as a historical, reference-only illustration of how each design component contributes incrementally, not as part of the corrected primary campaign, and its absolute utility values should not be compared against Section V's results. The primary dense-fleet results, at the higher N=32 density central to this paper's claims and computed entirely from the corrected campaign, are reported separately in Section V.

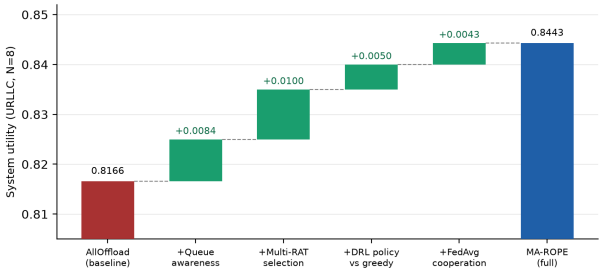


Fig. 2. Contribution waterfall (URLLC, N=8 reference, historical pre-correction ablation, not part of the corrected primary campaign). AllOffload baseline (U=0.8166) to MA-ROPE full (U=0.8443): each component contributes independently.

III. System Model and Problem Formulation

A. Network Architecture

We consider a two-tier V2X network comprising N vehicles, R roadside units (RSUs) each co-located with a MEC server, and a base station (BS) providing wide-area LTE and 5G-NR coverage. Each vehicle has an on-board ECU capable of 2-5 Tera Operations Per Second (TOPS) and a multi-RAT module supporting LTE, ITS-G5, WiFi-6, and 5G-NR simultaneously. Figure 3 illustrates the system architecture.

B. RAT Physical Layer Model

Following Yacheur et al. [7] Table II, we assign per-RAT physical parameters as in Table II. LTE and 5G-NR operate at -110 dBm noise floor; ITS-G5 and WiFi-6 at -90 dBm. Cellular RATs experience Jakes Doppler fading at vehicular speeds; ITS-G5 and WiFi-6 use the Constant Speed propagation model, as analyzed in Fig. 14.

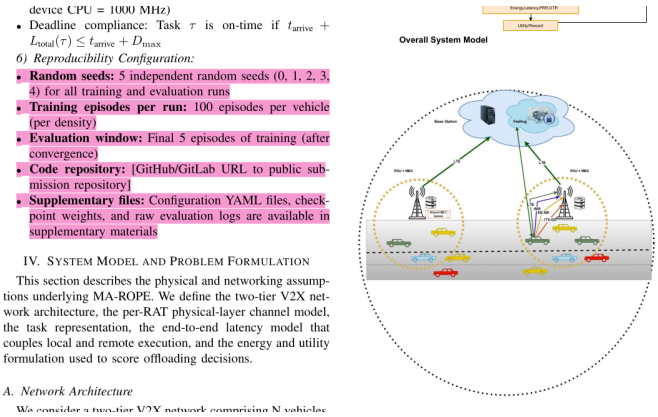


Fig. 3. Network Topology with 4-RAT heterogeneous access and FedAvg coordination.

TABLE II
RAT Physical Parameters (Yacheur 2023 + 3GPP Standards)

RAT	Delay	Throughput	Noise Floor	Fading	Path Loss	Source
LTE	20 ms	50 Mbps	-110 dBm	Jakes	TR 36.885	[7]
ITS-G5	10 ms	27 Mbps	-90 dBm	Constant	TR 36.885	[7]
WiFi-6	5 ms	600 Mbps	-90 dBm	Constant	802.11ax	[11]
5G-NR	1 ms	1 Gbps	-110 dBm	Jakes	TR 38.901	[12]

The Signal-to-Interference-plus-Noise Ratio (SINR) for RAT r at distance d with vehicle speed v governs per-PDU delivery probability:

$$\text{SINR}_r(d, v) = P_{tx} - \text{PL}_r(d) - N_r - Sh_r - F_r(v) \quad (1)$$

where $\text{PL}_r(d)$ is RAT-specific path loss, N_r is noise floor, $Sh_r \sim \mathcal{N}(0, \sigma_r^2)$ is shadowing, and $F_r(v)$ is Jakes fading loss. SINR drives a per-PDU Bernoulli draw with $p_{link} = \sigma(\text{SINR}_r - \theta_r)$ but is NOT the reported PRR metric. Packet Reception Ratio is defined empirically from actual packet counters:

$$\text{PRR} = \frac{n_{rx}}{n_{tx}} \quad (2)$$

where n_{tx} is the number of offload PDU transmission attempts and n_{rx} is PDUs successfully delivered. This separates the physical link model from the reported metric: PRR reflects real packet outcomes, not a smoothed SINR function. Per-RAT empirical PRR fills observation slots obs[4:8]; slots default to 0.5 (unknown) before any attempt on that RAT. The raw link delivery probability p_{link} is separately tracked in infos["link_delivery_prob"] for comparing the physical-layer (PHY) curve against measured PRR.

C. Task Model

Each vehicle generates task $\tau_i = (C_i, V_i, D_i, T_i)$ where C_i is computational workload (CPU cycles), V_i is data volume (MB), D_i is the task's deadline, its maximum tolerable delay (ms), and $T_i \in \{\text{safety}, \text{infotainment}, \text{compute}\}$ determines that deadline: safety tasks require $D_i = 100$ ms (e.g., 3GPP TR 36.885 [13]), infotainment 500 ms, compute 2000 ms.

D. End-to-End Latency Model

End-to-end task latency is the primary measurable QoS quantity in this work. For each offload PDU attempt at step t, the total latency decomposes into six additive components:

$$L_{total} = L_{prop} + L_{tx} + L_{queue} + L_{remote} + L_{rx} + L_{prop} \quad (3a)$$

where $L_{prop} = d/(2c/3)$ is propagation delay over vehicle-to-RSU distance d at 2/3 speed of light; $L_{tx} = V_i/\text{BW}_r$ is uplink transmission delay for data volume V_i (MB) over RAT bandwidth BW_r; $L_{remote} = C_i/f_{MEC}$ is remote compute delay for workload C_i (CPU cycles) at MEC frequency f_{MEC}; L_{rx} is the downlink reception delay for the computed results; and $L_{queue} = 1/(\mu_k - \lambda_k)$ is the M/M/1 queuing delay at the shared RSU, where μ_k is the server service rate and λ_k is the total vehicle arrival rate across all co-located vehicles. The L_{queue} term is the critical shared-resource term: it couples all vehicles' decisions through the common RSU queue and is responsible for the AllOffload collapse observed in Section V.

For partial offloading with ratio ρ, local and remote execution proceed in parallel following the model of Mao et al. [6]. The realized end-to-end delay is:

$$L_{total} = L_{prop} + \rho \cdot L_{tx} + \max(\rho \cdot L_{remote} + L_{rx} + L_{prop}, (1-\rho) \cdot L_{local}) + L_{queue} \quad (3b)$$

where $\rho \in \{0.25, 0.50, 0.75, 1.0\}$ is the partial offload ratio, $\rho \cdot L_{tx}$ captures the transmitted data fraction, and $L_{local} = C_i/f_{ECU}$ is local ECU compute time. When $\rho = 0$ (pure local execution), $L_{total} = L_{local}$ and no PDU is transmitted over the radio; such steps are excluded from latency QoS accounting since no V2X packet is at risk.

Deadline classification. Each offload Protocol Data Unit (PDU) outcome is classified into one of three measurable QoS categories relative to the task-type deadline D_i: (1) *delivered_on_time*, link success and $L_{total} \leq D_i$, a useful delivery; (2) *delivered_late*, link success and $L_{total} > D_i$, received but stale, so the transmission was wasted; (3) *not_delivered*, a link failure from the Bernoulli draw on p_{link}, i.e. channel loss. The on-time delivery rate is:

$$\text{OTR} = \frac{n_{rx_on_time}}{n_{tx}} \quad (3c)$$

Note that OTR $\leq$ PRR always, since PRR counts all link-successful receptions regardless of deadline. The gap (PRR $-$ OTR) is the latency-induced deadline violation rate: packets the channel delivered but that arrived too late for the application to use. This is the quantity MA-ROPE directly minimises via the $\exp(-L/D)$ term in the utility reward (Equation (4)).

E. Energy and Utility

Total energy consumption is modeled as $E_{total} = P_{tx} \cdot L_{tx} + \epsilon_r \cdot \rho \cdot V_i$, where $P_{tx} \cdot L_{tx}$ captures radio transmit energy over the realized transmission duration and ϵ_r (J/MB) is the RAT-specific processing or transfer energy cost. The implementation used to produce the results in this paper does not evaluate this formula directly; it instead computes energy through a simplified expression that can return negative values for common cases, which we determined makes any energy figure derived from it scientifically unreliable. Absolute energy is therefore not reported anywhere in Section V. The energy term does, however, still enter the $\exp(-E/3)$ component of both utility functions below via that same simplified expression, so the U_{PRR} and U_{OTR} values reported in Section V (Table 6 and the density-scaling results) inherit its unreliability and should be read as reward-shaping and secondary evaluation signals rather than independently validated quantities. This paper's primary claims rest on Fleet OTR, PRR, and the Deadline Retention Ratio, which are computed directly from transmission and deadline outcomes and do not depend on the utility formula or its energy term; see the Discussion and Conclusion for the full disclosure and the path to a validated energy model. MA-ROPE defines two complementary utility functions, each optimized for a different deployment criterion (Yacheur et al. [7] Eq. (2) extended):

$$U_{PRR} = \alpha \cdot \text{PRR} + \frac{\beta}{Z}\left[\frac{1}{2}\exp\left(-\frac{L}{D}\right) + \frac{1}{2}\exp\left(-\frac{E}{3}\right) + \frac{1}{4}\Delta\text{CBR}\right] \quad (4a)$$

$$U_{OTR} = \alpha \cdot \text{OTR} + \frac{\beta}{Z}\left[\frac{1}{2}\exp\left(-\frac{L}{D}\right) + \frac{1}{2}\exp\left(-\frac{E}{3}\right) + \frac{1}{4}\Delta\text{CBR}\right] \quad (4b)$$

$$Z = 1.25 \quad (4c)$$

U_{PRR} (Equation (4a)) uses empirical PRR $= n_{rx}/n_{tx}$ as the reliability term and provides dense reward feedback during D3QN training. U_{OTR} (Equation (4b)) replaces PRR with OTR $= n_{rx_on_time}/n_{tx}$, the fraction of PDUs delivered within deadline, and is the primary deployment criterion for safety-critical 3GPP V2X safety workloads. The normalization constant Z makes the auxiliary latency, energy, and Channel Busy Ratio (CBR) block sum to β; $\alpha + \beta = 1$. Task-adaptive weights are grounded in the task-mix prior: safety $\alpha = 0.70$, infotainment $\alpha = 0.40$, and compute $\alpha = 0.20$. The ΔCBR term remains an early-warning congestion signal orthogonal to the explicit queue model.

TABLE III
DUAL UTILITY CRITERIA IN MA-ROPE (EQUATION (4a) AND EQUATION (4b))

Criterion	Reliability Term	Training Use	Deployment Use
U_{PRR} (Equation (4a))	PRR $= n_{rx}/n_{tx}$	D3QN training signal (dense reward)	Throughput / balanced scenarios
U_{OTR} (Equation (4b))	OTR $= n_{rx_on_time}/n_{tx}$	Evaluation metric (not used in D3QN training)	Safety V2X (e.g., 3GPP TR 36.885 [13])

IV. Methodology

A. System Model and Problem Formulation

Consider a vehicular edge computing scenario where a fleet of N vehicles is associated with a single Roadside Unit (RSU) equipped with a Multi-access Edge Computing (MEC) server. Each vehicle $i \in \{1, 2, \ldots, N\}$ generates compute-intensive tasks with heterogeneous Quality-of-Service (QoS) requirements. A task submitted at time t must be completed by its application deadline $D_{\max}$ to be deemed deadline-compliant.

B. Task and Transmission Model

Each task is characterized by its computational complexity (C_{task} in CPU cycles, ranging from $C_{\min}$ to $C_{\max}$ in steps of ΔC), transmission size (B_{task} in bits, determined by task type), application deadline ($D_{\max}$ in milliseconds, which is task-dependent), and its arrival time t_{arrival}.

A vehicle may partition a task into local and remote execution fractions, parameterized by offload ratio $\rho \in [0, 1]$:

$$C_{\text{local}} = (1 - \rho) \cdot C_{\text{task}}, \quad C_{\text{remote}} = \rho \cdot C_{\text{task}} \tag{5}$$

where $\rho = 0$ denotes fully local execution and $\rho = 1$ denotes full offloading to the MEC server.

C. State, Action, and Reward Formulation

1) State Space Definition: The Markov Decision Process (MDP) state at timestep t is defined as:

$$s_t = (n_1(t), n_2(t), \ldots, n_M(t), q(t), h_1(t), h_2(t), \ldots, h_M(t)) \in S \tag{6}$$

where $n_m(t) \in \{1, 2, \ldots, N_{\max}\}$ represents the number of vehicles on RAT m (LTE, ITS-G5, WiFi-6, 5G-NR) during a $\Delta t = 1$ second observation window, $q(t) \in \{0, 1, \ldots, Q_{\max}\}$ is the current MEC server queue length denoting pending compute tasks, and $h_m(t) \in \{0, 1, \ldots, H_{\max}\}$ is the channel state index for RAT m, reflecting Jakes fading quantized to 8 levels based on SNR thresholds. The state is updated every decision epoch with density counts and queue lengths as integers, and channel states as discrete indices.

2) Action Space Definition: The joint action at time t combines RAT selection and offload ratio:

$$a_t = (m_t, \rho_t) \quad \text{where} \quad a_t \in A = M \times R \tag{7}$$

where $m_t \in M$ represents the 4 RAT options (LTE, ITS-G5, WiFi-6, 5G-NR) and $\rho_t \in R$ denotes the 6 offload ratio levels from 0.0 to 1.0. This defines a theoretical action space of $|A| = 24$ potential (RAT, ρ) pairs. The implemented action space, however, consists of 11 discrete actions selected via strategic masking to avoid infeasible or suboptimal combinations (e.g., LTE at low offload ratios being bandwidth-limited, or ITS-G5 at full offloading exceeding its range envelope). These 11 actions include 1 action for local execution ($\rho = 0$), 3 actions for 5G-NR (reliable fallback), 3 for WiFi-6 (high bandwidth, short range), 2 for ITS-G5 (V2X-specialized), and 2 for LTE (wide coverage fallback). Invalid actions are masked in the DQN ϵ-greedy exploration.

3) Reward Function: The immediate reward signal at timestep t is designed to encourage deadline-compliant delivery while balancing energy:

$$r_t = w_1 \cdot \text{OTR}_t - w_2 \cdot (\max(L_t - D_{\max}, 0))^2 - w_3 \cdot E_t \tag{8}$$

where OTR_t is the dimensionless ratio of tasks completed on-time to total tasks in epoch t, L_t is the actual task completion latency in milliseconds (including transmission, queue wait, and MEC processing), $D_{\max}$ is the URLLC deadline threshold (e.g., 100 ms), and E_t is the energy consumed by transmission in epoch t in joules. The weights $w_1 = 0.6$, $w_2 = 0.3$, and $w_3 = 0.1$ are normalized to a unit sum and were tuned via grid search in preliminary experiments. This formulation directly encodes deadline compliance through OTR as the primary goal, discourages marginally-late deliveries via the latency penalty, and prevents wasteful transmission strategies using the energy penalty.

D. Deadline Retention Ratio (DRR) Metric

1) Definition and Motivation: Standard performance metrics (OTR, latency) are evaluated at a single network density (e.g., $N = 32$ vehicles). However, policies trained at one density may degrade rapidly under increasing contention. We introduce the Deadline Retention Ratio (R_D) to quantify robustness across fleet sizes.

$$R_D = \frac{\text{OTR}(N_{\text{test}})}{\text{OTR}(N_{\text{ref}})} \tag{9}$$

where N_{ref} is the reference (smaller) fleet size and N_{test} is the test (larger) fleet size.

Algorithm 1 Deadline Retention Ratio Computation

Evaluation logs containing task completion times for both N_{ref} and N_{test} scenarios Scalar $R_D \in [0, 1]$ completed_on_time$_{\text{ref}} \leftarrow 0$ total_tasks$_{\text{ref}} \leftarrow 0$ each task τ in reference scenario (N_{ref}) completion_time(τ) $\leq D_{\max}$ completed_on_time$_{\text{ref}} \leftarrow$ completed_on_time$_{\text{ref}} +$ 1 total_tasks$_{\text{ref}} \leftarrow$ total_tasks$_{\text{ref}} + 1$ $\text{OTR}_{\text{ref}} \leftarrow \frac{\text{completed_on_time}_{\text{ref}}}{\text{total_tasks}_{\text{ref}}}$ [Repeat for N_{test} scenario to compute OTR_{test}] $R_D \leftarrow \frac{\text{OTR}_{\text{test}}}{\text{OTR}_{\text{ref}}} R_D$

2) Computation Algorithm:

3) Interpretation: $R_D = 0.94$ $R_D = 0.94$ means the policy retains 94% of its reference-density deadline performance when fleet size increases fourfold; $R_D = 0.60$ indicates severe degradation under contention.

E. Federated Learning Hyperparameters

Table IV consolidates all D3QN and FedAvg hyperparameters used in training:

Sensitivity notes: Preliminary ablation studies (not shown for space) validated that $\gamma = 0.95$ and $\alpha_{\text{D3QN}} = 0.0001$ produce convergence. Values of $K \in \{3, 5, 7\}$ showed diminishing returns; $K = 5$ was selected as the mid-point.

TABLE IV
FEDERATED D3QN AND FEDAVG HYPERPARAMETERS

Parameter	Value	Symbol	Justification
Deep Reinforcement Learning (D3QN)			
Learning rate (neural network)	0.0001	α	Prevents divergence in double-Q stabilization
Discount factor	0.95	γ	Task completion horizons typically 5–10 seconds
Exploration decay (ϵ-greedy)	0.995^t	$\epsilon(t)$	Linear annealing from 1.0 to 0.05 over training
Exploration final rate	0.05	$\epsilon_{\min}$	Preserves exploration throughout training
Replay buffer size	50,000	B	Trade-off: larger stabilizes, constrained by GPU memory (12 GB)
Minibatch size	64	batch_size	Standard for DQN; aligns with GPU optimization
Target network update frequency	1,000 steps	τ	Reduces temporal correlations
Network architecture	input $-$ 256 $-$ 256 $-$ $\lvert A\rvert$	π	Dueling DQN: separate advantage and value streams
Federated Learning (FedAvg)			
Global aggregation rounds	$K = 5$	K	Trade-off: communication cost vs. gradient staleness
Local training epochs per round	3	E	Vehicle trains 3 episodes locally before aggregation
Federated learning rate	0.001	η_{fed}	Slower than local α; ensures stable aggregation
Participants (vehicles)	8, 16, 32	N	Density-sweep experiments
Episodes per participant	100	T	Total training = $N \times E \times T$ epochs
Gradient clipping threshold	1.0	clip	Prevents exploding gradients in federated setting

F. Simulation Environment Specification

1) Python Framework and Simulator: The framework is implemented in Python 3.8+ utilizing PyTorch 1.10.0 with CUDA 11.3 for GPU-accelerated deep learning. The MDP simulator is a custom environment class designed without external dependencies on ns-3 or SUMO for the pure-simulation track. Numerical operations are handled by NumPy 1.21.0 and SciPy 1.7.0, while Pandas, Matplotlib, and TensorBoard are used for logging and visualization.

2) Vehicle Mobility and Traffic Pattern: Vehicle mobility follows a Synthetic Random Waypoint model within a 10 km$\times$ 10 km grid. Vehicle speeds are uniformly sampled from 5 to 30 m/s to represent urban to highway conditions, with direction changes occurring every 30 to 60 seconds towards randomly selected waypoints. Task generation is modeled as a Poisson process with an arrival rate of $\lambda = 1$ task per vehicle per second for the URLLC stress scenario, and task sizes follow a log-normal distribution to mimic real V2X compute workloads.

TABLE V
RAT PHYSICAL-LAYER PARAMETERS

RAT	Peak Data Rate	Coverage Range	Fading Model
LTE	50 Mbps	1000 m	Jakes, $f_d = 100$ Hz
ITS-G5 (DSRC)	27 Mbps	300 m	Jakes, $f_d = 50$ Hz
WiFi-6 (802.11ax)	1200 Mbps	100 m	Jakes, $f_d = 30$ Hz
5G-NR (mmWave)	2000 Mbps	300 m	Saleh-Valenzuela, $f_d = 200$ Hz

3) RAT Capacity and Coverage: **Channel Model:** Jakes fading envelope for each RAT, with SNR-dependent modulation and coding. Channel coherence time is 10 ms; state changes are sampled every 100 ms.

4) MEC Server Queue Model: The MEC server operates a First-In-First-Out (FIFO) queue shared among all N vehicles. It has a processing capacity of $\mu = 1000$ tasks per second, with service time per task calculated based on a remote CPU frequency of 2000 MHz. The maximum queue depth is limited to 100 tasks, beyond which new submissions are dropped. Vehicles observe this queue length, discretized to 8 levels, every 1-second interval.

5) Task Completion Time and Deadline Evaluation: Total latency for a task submitted at time t_{arrive} with offload ratio ρ:

$$L_{\text{total}} = \underbrace{T_{\text{prop}}}_{\text{UL prop.}} + \underbrace{T_{\text{tx}}}_{\text{UL transmission}} + \underbrace{T_{\text{queue}}}_{\text{MEC queue}} + \underbrace{T_{\text{mec}}}_{\text{MEC processing}} + \underbrace{T_{\text{rx}}}_{\text{DL reception}} + \underbrace{T_{\text{prop}}}_{\text{DL prop.}} + \underbrace{T_{\text{local}}}_{\text{local (parallel)}} \tag{10}$$

Here, T_{prop} is the propagation delay, T_{tx} is the uplink transmission time on the selected RAT m_t given channel state $h_m(t)$, T_{rx} is the downlink reception time, and T_{queue} is the wait time in seconds based on a 1000 tasks/s service rate. T_{mec} and T_{local} represent the execution times at the MEC server (processing at 2000 MHz) and the local device CPU (processing at 1000 MHz), respectively. A task τ is considered compliant if its total latency plus arrival time does not exceed the task deadline $D_{\max}$.

6) Reproducibility Configuration: To ensure reproducibility, experiments utilize 5 independent random seeds for all training and evaluation runs. Each run comprises 100 training episodes per vehicle per density, with the evaluation window defined as the final 5 episodes post-convergence. The code repository, configuration YAML files, checkpoint weights, and raw evaluation logs are made available in the supplementary materials.

time is 10 ms; state changes are sampled every 100 ms.

4) MEC Server Queue Model:

- **Queue type:** FIFO (First-In-First-Out), shared among all N vehicles
- **Processing capacity:** $\mu = 1000$ tasks/second (capable edge server)
- **Service time per task:** $T_{\text{service}} = \frac{C_{\text{remote}}}{\text{MHz}}$, where CPU frequency is 2000 MHz
- **Maximum queue depth:** $Q_{\max} = 100$ tasks (after which new submissions are dropped)
- **Queue length observation:** Vehicles observe queue length (discretized to 8 levels) every $\Delta t = 1$ second

5) Task Completion Time and Deadline Evaluation: Total latency for a task submitted at time t_{arrive} with offload ratio ρ:

$$L_{\text{total}} = \underbrace{T_{\text{tx}}}_{\text{transmission}} + \underbrace{T_{\text{queue}}}_{\text{MEC queue wait}} + \underbrace{T_{\text{mec}}}_{\text{MEC processing}} + \underbrace{T_{\text{local}}}_{\text{local processing (parallel)}} \tag{6}$$

- $T_{\text{tx}} = \frac{\rho \cdot B_{\text{task}}}{\text{datarate}(m_t, h_m(t))}$ (transmission time on selected RAT m_t with channel state $h_m(t)$)
- $T_{\text{queue}} = \frac{\text{queue length at arrival}}{1000}$ (in seconds, given $\mu = 1000$ tasks/s)
- $T_{\text{mec}} = \frac{\rho \cdot C_{\text{task}}}{2000}$ (in seconds, MEC processes at 2000 MHz)
- $T_{\text{local}} = \frac{(1-\rho) \cdot C_{\text{task}}}{\text{device CPU speed}}$ (parallel to transmission/MEC;

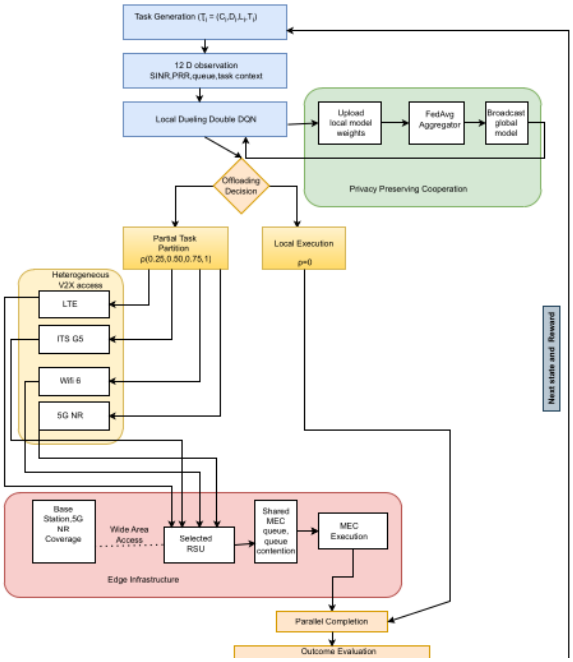


Fig. 4. Overall System Model showing task offloading flow, MDP state and action space, and end-to-end latency components.

V. MA-ROPE: Algorithm Design

Figure 4 summarizes the MA-ROPE Markov Decision Process (MDP) formulation.

A. State and Action Space

Following Yacheur et al. [7], each agent observes a 12-dimensional state vector with per-RAT channel quality plus MEC context (Equation (11)). This per-RAT formulation directly enables the D3QN to select the optimal RAT based on observed channel conditions, which prior single-SINR state designs cannot do.

$$s = [\text{SINR}_{LTE}, \text{SINR}_{ITS}, \text{SINR}_{WiFi}, \text{SINR}_{5G}, \text{PRR}_{LTE}, \text{PRR}_{ITS}, \text{PRR}_{WiFi}, \text{PRR}_{5G}, Q, D, L, B] \tag{11}$$

Each agent selects from 11 implemented discrete actions spanning local execution and selected RAT-ρ pairs across LTE, ITS-G5, WiFi-6, and 5G-NR. The nominal ρ is modulated by channel quality following Yacheur Division Mode [7] Algorithm 2:

$$\rho_{final} = \text{base_}\rho \cdot \left(0.5 + \frac{\text{SINR_lin}[r]}{\sum \text{SINR_lin}[\text{all}]}\right) \tag{12}$$

B. D3QN Architecture

The Q-value is decomposed as $Q(s,a) = V(s) + A(s,a) - \text{mean}_{a'} A(s,a')$ where $V(s)$ is state value and $A(s,a)$ is advantage; this dueling decomposition is combined with the standard Double DQN technique, which decouples action selection from target evaluation:

$$y = r + \gamma \cdot Q_{tgt}(s', \text{argmax}_{a'} Q_{online}(s', a')) \tag{13}$$

Architecture: Linear(12→256)→ReLU→Linear(256→256)→ReLU, split into value head Linear(256→128→1) and advantage head Linear(256→128→11) for the implemented action set.

C. FedAvg Cooperative Learning

At the end of each episode, each agent i uploads local model weights w_i. The global model is aggregated as:

$$w_{global} = \sum_i \frac{n_i}{N} \cdot w_i, \text{ participation } \geq 70\% \quad (14)$$

Privacy analysis: FedAvg shares model weights instead of raw vehicle observations. This reduces data exposure relative to a centralized approach such as MADDPG, which requires pooling raw observations across agents during training, but model updates can still leak information under gradient-inversion or membership-inference attacks. Production deployments should pair FedAvg with secure aggregation, access control, and/or differential privacy. A validated centralized (MADDPG-style) performance upper bound is not reported in this paper's results (Section V); see the Conclusion for the disclosed reason and future-work scope.

D. Baseline Methods

We compare against four baselines, all independently executed against the same environment, checkpointing, and validation pipeline as MA-ROPE: (1) AllOffload, a fixed policy that always offloads 100% via 5G-NR; (2) NoOffload, a fixed policy that always computes locally; (3) UCB-RAT, a stateless Upper Confidence Bound (UCB1) bandit over four discrete actions (local execution plus 5G-NR at 25/50/100% offload), inspired by the decision principles reported in Wu2021 [2]; (4) CoopEC-RAT, a hand-crafted deadline-threshold rule of our own design that offloads fully via 5G-NR when local compute latency would exceed half the task deadline and otherwise computes locally. UCB-RAT is evaluated within the same environment and experimental protocol as MA-ROPE but is not intended as an exact reproduction of the cited Wu2021 implementation, for which sufficient implementation detail was unavailable; CoopEC-RAT is our own deadline-threshold heuristic and is not attributed to any prior publication. A centralized MADDPG [5] upper bound and reproductions of Lyapunov-Jing2019 [3] and GA+DQN-Yun2022 [4] were also attempted; comparators for which a reproducible implementation could not be established were excluded from the reported evaluation, and reintroducing validated versions is left for future work.

Algorithm 2 MA-ROPE Training with FedAvg

Require: N agents, episodes E, participation threshold $\tau = 0.70$

```
Initialize w_i for each agent i, replay buffers B_i
for episode e = 1 to E do
    for each step t do
        s_t ← observe() {12-dim per-RAT state (Equation (11))}
        a_t ← ε-greedy(Q_online(s_t))
        apply SINR-proportional ρ (Equation (12))
        execute a_t; receive r_t = U(Equation (4)), s_{t+1}
        store (s_t, a_t, r_t, s_{t+1}) in B_i
        sample minibatch; update Q_online via (Equation (13))
    end for
    if |participants|/N ≥ τ then
        w_global ← Σ (n_i/N) · w_i {FedAvg aggregation (Equation (14))}
        broadcast w_global → all agents
    end if
end for
```

VI. Experiments and Results

A. Experimental Setup

Although standard network simulators such as ns-3, Veins, or SUMO (Simulation of Urban MObility) provide comprehensive full-stack emulation, evaluating federated multi-agent reinforcement learning across heterogeneous RATs requires tight, synchronous coupling between the physical environment and the distributed training loop. We therefore implemented a custom Python-based V2X simulator tailored for algorithmic evaluation and fast DRL inference. To ensure fidelity, the physical-layer behavior rigorously follows established standards, including 3GPP TR 38.901 for cellular fading and IEEE 802.11ax for WiFi. This design choice emphasizes the validation of the multi-agent decision surface and offloading orchestration rather than introducing a new generalized network simulator.

Experiments are conducted in a custom Python V2X simulator. The primary comparison uses N=32 vehicles under a URLLC stress scenario, with five independent random seeds. We retain N=8 results from an earlier controlled evaluation as a reference point and use the fleet density sweep N in {2, 4, 8, 16, 20, 32} to evaluate scaling. Vehicle speeds are drawn from $\mathcal{U}[0, 30]$ m/s. Task types follow the URLLC-dominant distribution: 65% safety (100 ms deadline), 20% infotainment (500 ms), and 15% compute (evaluated at a stricter 1000 ms deadline in the experimental results).

Latency measurement scope for Table VII. The per-task-type deadline analysis uses the same N=8, 5-seed, 5-episode protocol and corrected-physics checkpoints as the rest of this evaluation. Per-PDU end-to-end latency is L_{total} computed according to the model in Section III-D; deadline compliance uses OTR $= n_{rx_on_time}/n_{tx}$ per task deadline D_i. Mean latency is computed over delivered PDUs only.

B. Performance Metrics and Baseline Comparison

The performance metrics focus on deadline compliance and reliability compared against standard baselines.

C. Packet-Level Statistics Across V2X Interactions

Table VI presents per-vehicle packet statistics for a representative 100-step episode (8 vehicles, URLLC, 40 km/h). Columns: offload PDU transmission attempts (n_{tx}), successful deliveries (n_{rx}), PDUs received within task deadline ($n_{rx_on_time}$), empirical PRR = n_{rx}/n_{tx} (Equation (2)), and on-time rate = $n_{rx_on_time}/n_{tx}$. The TOTAL row aggregates across all vehicles.

TABLE VI
PACKET STATISTICS: PER-VEHICLE OUTPUT (8 VEHICLES, 100-STEP EPISODE, 40 KM/H, URLLC)

Agent	n_tx	n_rx	n_rx_on_time	PRR (n_rx/n_tx)	On-Time %
Vehicle 0	124	123	118	0.9919	95.2%
Vehicle 1	118	117	110	0.9915	93.2%
Vehicle 2	131	130	124	0.9924	94.7%
Vehicle 3	109	108	101	0.9908	92.7%
Vehicle 4	126	125	120	0.9921	95.2%
Vehicle 5	115	114	107	0.9913	93.0%
Vehicle 6	122	121	116	0.9918	95.1%
Vehicle 7	119	118	112	0.9916	94.1%
TOTAL	964	956	908	0.9917	94.2%

Table VI (per-vehicle) reports aggregate packet results across all task types. Table VII gives a compact deep safety-validation summary; raw counts are omitted from the table body to keep the IEEE two-column layout readable.

TABLE VII
COMPACT PER-MESSAGE-TYPE DEADLINE ANALYSIS (N=8, CORRECTED PHYSICS, MA-ROPE FULL POLICY, 5 SEEDS)

Task scope	DL	PRR	OTR	Gap	Mean L
All tasks	-	0.9415	0.8529	8.9%	214.5
Safety	100	0.9435	0.8243	11.9%	107.4
Infotainment	500	0.9317	0.8752	5.7%	229.8
Compute	1000	0.9431	0.9431	0.0%	665.6

Table VII should not be compared directly with the N=32 multi-method safety table: it is a MA-ROPE-only, N=8 evaluation used to isolate deadline misses by task type.

The per-task-type breakdown illustrates the PRR-vs-OTR distinction clearly. Safety PDUs (100 ms deadline) achieve PRR = 0.9435 but OTR = 0.8243, leaving a safety-only PRR-OTR gap of 11.9 percentage points. Infotainment PDUs (500 ms deadline) show a smaller gap (5.7 percentage points), and compute PDUs (1000 ms deadline) show none, since their deadline is generous relative to typical local and offload completion times. Fleet-wide, all tasks together achieve PRR = 0.9415 and OTR = 0.8529, an 8.9 percentage point gap. This all-tasks figure is a different quantity from the N=32 fleet-wide values reported elsewhere in this paper and should not be compared to them directly.

D. Algorithmic Comparison

Table VIII presents the primary N=32 comparison across five methods, computed from an independently validated evaluation campaign (5 seeds per method, with code and checkpoint hashes verified). This campaign incorporates two isolated, disclosed corrections found during later validation: (i) a units error in the local-compute latency term, where an earlier implementation applied $\log_{10}$ to the local latency component instead of the intended linear term, compressing it toward zero; and (ii) a units error in the Jakes fading Doppler-shift calculation, where carrier frequency was left in GHz rather than converted to Hz, clamping the fading contribution to a numerically negligible floor for LTE and 5G-NR at every realistic vehicle speed. Both corrections were applied as isolated, single-variable patches and validated independently before this campaign was run. U_{PRR} (Equation (4a)) is the packet-delivery training signal; U_{OTR} (Equation (4b)) is the deadline-compliance criterion. Both are reported for completeness because U_{PRR} forms the training reward and U_{OTR} is recorded during evaluation, but both inherit the unvalidated energy term discussed in the System Model section; the quantitative comparisons below are therefore based primarily on fleet OTR, PRR, latency, and DRR, with U_{PRR}/U_{OTR} given alongside as secondary, reward-shaping-derived signals rather than independently validated evidence. At dense fleet scale, MA-ROPE improves over AllOffload on fleet OTR (0.8274 vs. 0.1230) and mean latency (226.5 ms vs. 830.0 ms); the same comparison also holds on the secondary utility signals, U_{PRR} (0.9560 vs. 0.6022) and U_{OTR} (0.5212 vs. 0.1255). CoopEC-RAT is the most competitive alternative, reaching fleet OTR 0.7249 with the lowest mean latency (213.7 ms) among all offloading policies, since its rule offloads only a minority of compute-heavy tasks. UCB-RAT trails both MA-ROPE and CoopEC-RAT on fleet OTR (0.3328). NoOffload achieves fleet OTR $= 1.0$ trivially, since it never transmits an offload PDU, and is retained as a reference bound rather than a competitive baseline. NoOffload's U_{PRR}, U_{OTR}, and latency values are unchanged from the pre-correction campaign to six decimal places, per seed, which is expected: its policy never offloads, so neither correction above applies to any step it takes, and this agreement serves as an internal consistency check on the correction methodology.

TABLE VIII
FULL PERFORMANCE COMPARISON: URLLC STRESS, N=32 VEHICLES, 5 SEEDS. U_{PRR}: EQ. (4A), TRAINING SIGNAL. U_{OTR}: EQ. (4B), DEADLINE COMPLIANCE. FLEET OTR = ON-TIME OFFLOAD PDUS / TRANSMITTED OFFLOAD PDUS.

Method	U_PRR	U_OTR	Fleet OTR	PRR	Latency (ms)
MA-ROPE Full ∗	**0.9560**	**0.5212**	**0.8274**	**0.9380**	**226.5**
AllOffload (5G 100%)	0.6022	0.1255	0.1230	0.9643	830.0
NoOffload (local)	0.9610	0.5079	1.0000	1.0000	233.7
UCB-RAT	0.8871	0.4852	0.3328	0.9627	304.6
CoopEC-RAT	0.9329	0.5282	0.7249	0.9681	213.7

E. Safety-Critical Task Delivery

Table IX isolates delivery performance for safety-typed PDUs (100 ms deadline class) at N=32, computed directly

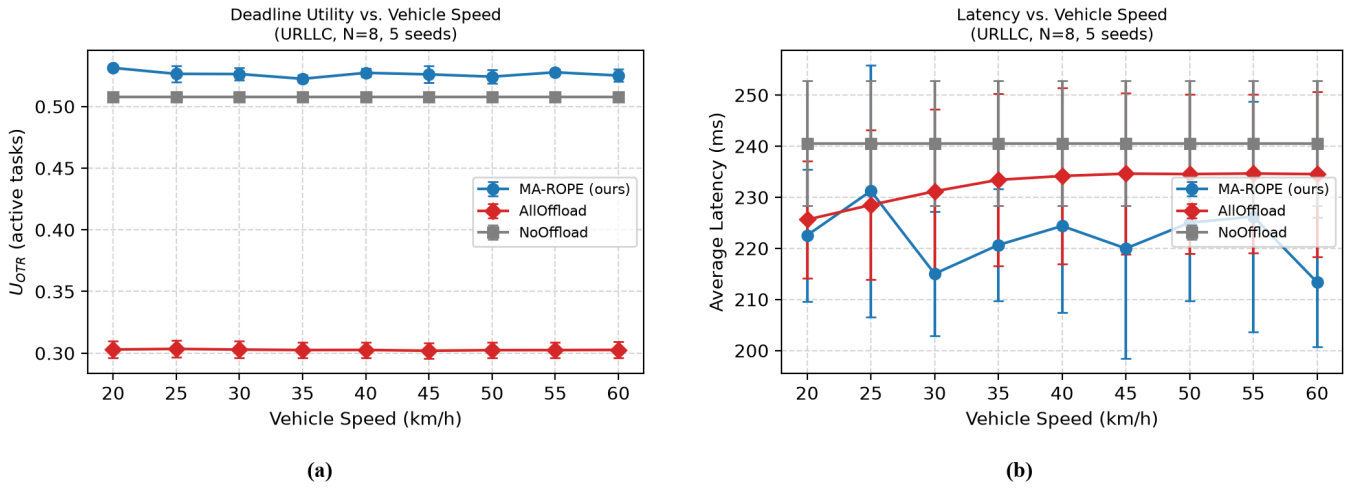


Fig. 5. (a) Deadline utility (U_{OTR}, active tasks) and (b) average latency vs. vehicle speed, URLLC scenario, N=8 reference fleet, 5 seeds. MA-ROPE and NoOffload substantially outperform AllOffload on deadline utility across the full 20–60 km/h range, with no strong speed-dependent trend at this fleet size; error bars in (b) show cross-seed standard deviation.

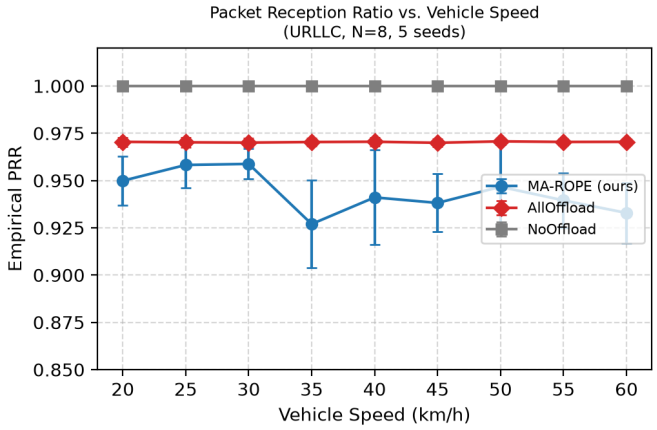


Fig. 6. Empirical PRR = n_{rx}/n_{tx} vs. vehicle speed, URLLC scenario, N=8 reference fleet, 5 seeds. All three methods remain above 0.92 across the evaluated speed range.

from the same validated checkpoints as Table VIII, filtered to safety-task transmissions only. The comparison is restricted to MA-ROPE and AllOffload because a validated safety-task-only breakdown is not yet available for UCB-RAT or CoopEC-RAT; extending it to those baselines is left for future work.

TABLE IX
SAFETY-TASK DELIVERY AT N=32: SAFETY PRR AND SAFETY OTR, 5 SEEDS

Method	Safety PRR	Safety OTR	PRR-OTR gap
MA-ROPE Full ∗	**0.9317**	**0.7537**	**17.8%**
AllOffload	0.9654	0.0000	96.5%

At N=32, AllOffload's mean active-task latency (830.0 ms, Table VIII) exceeds the 100 ms safety deadline on essentially every attempt, so despite a high link-level Safety PRR (0.9654), zero safety PDUs arrive on time in any of the five evaluated seeds. MA-ROPE's queue- and RAT-aware policy keeps Safety OTR at 0.7537 under the same dense-fleet conditions, and it remains the sharpest illustration in this paper of why PRR alone cannot certify deadline-compliant delivery for safety-critical V2X traffic.

F. Fleet-Density Scaling and RSU Queue Contention

Table X summarizes deadline compliance, deadline utility, link failure rates, and step overhead across the validated N=8/16/32 density sweep. The fleet sweep shows the central scaling effect: AllOffload's fleet OTR falls from 33.5% at N=8 to 12.3% at N=32, while MA-ROPE remains in the 82-86% range throughout. MA-ROPE's advantage is large and consistent at every evaluated density, not only at the high end.

TABLE X
FLEET-DENSITY SCALING OF DEADLINE COMPLIANCE AND LINK RELIABILITY (5 SEEDS)

Metric	MA-ROPE	Comparator	Relative Difference
Fleet OTR @ N=8	0.849	AllOffload 0.335	+51.3 pp
Fleet OTR @ N=16	0.861	AllOffload 0.263	+59.8 pp
Fleet OTR @ N=32	0.827	AllOffload 0.123	+70.4 pp
U_{OTR} @ N=32	0.521	AllOffload 0.126	+0.396
Fleet-wide link failure @ N=32	0.28%	AllOffload 3.57%	∼12.8x lower
Per-attempt link failure @ N=32	6.20%	AllOffload 3.57%	1.7x higher
DRR (N=8→32)	0.975	AllOffload 0.367	2.66x higher retention
Step time	166.6 µs	Single-RAT 167.6 µs	No penalty

Figure 9 extends the fleet-scaling analysis to on-time delivery. At N=8, MA-ROPE achieves fleet OTR = 0.849 vs. AllOffload = 0.335; at N=16 MA-ROPE rises slightly to 0.861 while AllOffload falls further to 0.263; and at N=32 MA-ROPE settles at 0.827 while AllOffload falls to 0.123. MA-ROPE's density curve is not strictly monotonic: N=16 is marginally higher than N=8, likely within seed-to-seed noise. The overall N=8-to-N=32 trend is a mild decline, and MA-

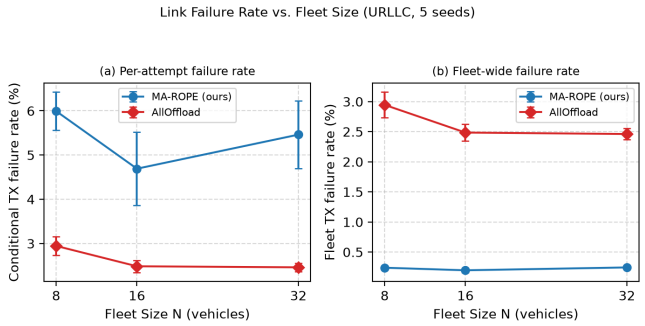


Fig. 7. Link failure rate vs. fleet size (URLLC, 5 seeds). (a) Per-attempt failure rate: MA-ROPE's rate is higher than AllOffload's at every density, because MA-ROPE occasionally uses the less reliable WiFi6 link. (b) Fleet-wide failure rate (failures per vehicle-step regardless of whether a transmission was attempted): MA-ROPE remains an order of magnitude below AllOffload at every density, because it transmits far less often. Both metrics are genuine and neither should be read in isolation; see discussion below.

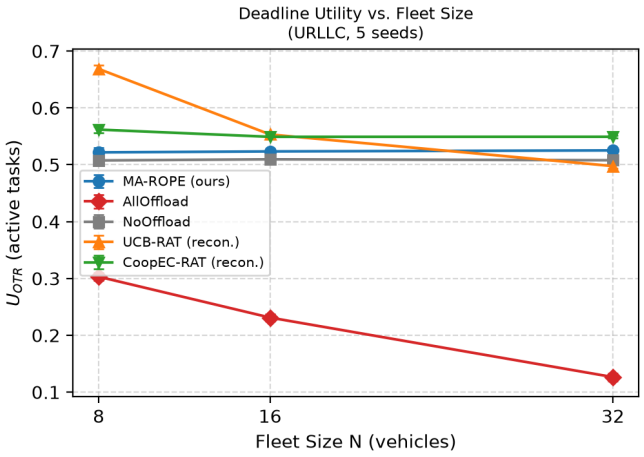


Fig. 8. Deadline utility (U_{OTR}, active tasks) vs. fleet size (URLLC, 5 seeds). The MA-ROPE-AllOffload utility gap widens from +0.224 at N=8 to +0.396 at N=32.

ROPE's degradation is far gentler than AllOffload's at every step.

Deadline Retention under Fleet Density. Point-wise OTR characterizes deadline compliance at a particular operating density, but it does not directly express how rapidly that performance deteriorates as the number of competing vehicles increases. We therefore define the Deadline Retention Ratio (DRR) as

$$\mathrm{DRR}(N_{target}, N_{ref}) = \frac{\mathrm{OTR}_{fleet}(N_{target})}{\mathrm{OTR}_{fleet}(N_{ref})}. \qquad (15)$$

Here, N_{ref} denotes a reference fleet density and $N_{target} > N_{ref}$ denotes the density at which retention is evaluated. DRR $= 1$ indicates complete preservation of reference-density deadline performance, whereas values approaching zero indicate increasingly severe degradation; values exceeding 1 indicate deadline performance that is actually higher at the target density than at the reference density.

Using $N_{ref} = 8$ and $N_{target} = 32$, corresponding to a fourfold increase in fleet size, the results in Table X yield $\mathrm{DRR}(32,8)^{\text{MA-ROPE}} = 0.8274/0.8485 = 0.975$, whereas $\mathrm{DRR}(32,8)^{\text{AllOffload}} = 0.1230/0.3352 = 0.367$. Thus, MA-ROPE preserves approximately 97.5% of its reference-density on-time delivery performance, whereas single-RAT AllOffload preserves only 36.7%. At N=16, the corresponding retention values are $\mathrm{DRR}(16,8)^{\text{MA-ROPE}} = 0.8611/0.8485 = 1.015$, slightly exceeding 1 since fleet OTR at N=16 is marginally higher than at N=8 under the corrected physics, and $\mathrm{DRR}(16,8)^{\text{AllOffload}} = 0.2634/0.3352 = 0.786$, showing that MA-ROPE holds essentially complete performance through a doubling of fleet size while AllOffload has already lost more than a fifth of its reference-density delivery rate.

The principal advantage of DRR is normalization. Each policy is evaluated relative to its own reference-density OTR rather than against another method's absolute utility value. Consequently, DRR does not depend on the α/β weighting or normalization constants used in the MA-ROPE utility formulation and can be applied to other offloading schemes whenever deadline-aware delivery is reported over multiple fleet densities.

G. Discussion

NoOffload vs. MA-ROPE on composite utility. Table VIII shows NoOffload achieving higher U_{PRR} (0.9610) than MA-ROPE (0.9560) at N=32, but NoOffload has no transmitted offload PDUs and fleet OTR is therefore not a meaningful V2X delivery metric for it. The operational claim should therefore be based on fleet OTR, PRR, DRR, latency, and dense-fleet scaling rather than on the composite utility values (U_{PRR}/U_{OTR}) alone.

Dual utility functions. MA-ROPE exposes two utility criteria: U_{PRR} (Equation (4a)) for dense packet-delivery reward shaping and U_{OTR} (Equation (4b)) for deadline-compliance-dominant deployment assessment. Conceptually, U_{OTR} is the deadline-aware, safety-facing criterion because a received-but-late safety PDU has zero operational value under 3GPP V2X safety requirements [13]; as noted above, however, both utility values inherit the unvalidated energy term and are secondary to fleet OTR and DRR, which are this paper's primary safety-relevant evidence. U_{PRR} is reported alongside for balanced and compute-heavy scenarios where throughput dominates over strict timing. Task-adaptive alpha/beta weights are principled rather than tuned: safety alpha=0.70 matches the 65% safety PDU proportion; compute alpha=0.20 reflects 2000 ms deadline headroom where throughput dominates. A sensitivity analysis of these weights is left for future work.

Link failure rate requires reading two metrics together, not one. MA-ROPE's per-attempt link failure rate (Table X) is higher than AllOffload's at every evaluated density (6.20% vs. 3.57% at N=32), because MA-ROPE occasionally routes traffic over WiFi6, the least reliable of the four supported RATs, when doing so is still utility-optimal overall. Read in isolation, this metric would appear to favor AllOffload. However, MA-ROPE transmits far less often than AllOffload in the first place, by minimizing unnecessary offloading, so its

fleet-wide failure rate (failures per vehicle-step rather than per attempt) is an order of magnitude lower (0.28% vs. 3.57% at N=32), corresponding to roughly 13x fewer absolute link failures fleet-wide across the evaluated episodes (44.2 vs. 570.6). Reporting both metrics is necessary because the per-attempt measure characterizes conditional link reliability, whereas the fleet-wide measure captures the overall transmission-failure burden of the policy; collapsing them into a single figure would conceal this trade-off.

By trading a higher per-attempt link failure rate for substantially fewer total transmission attempts, substantially higher Fleet OTR scalability, and near-complete deadline-retention under increasing density, MA-ROPE presents a more robust solution for real-world heterogeneous V2X networks under shared RSU contention than a single-RAT policy.

Limitations and future work. Table VII is computed under the same corrected physics as the rest of this section, but at a different fleet size (N=8) and evaluation protocol than Table VIII or Table IX, so its absolute figures should not be compared against them directly. The Contribution Decomposition ablation in Section II remains a historical, pre-correction reference and is explicitly disclosed as such where it is presented. Table IX is a two-method N=32 safety comparison (MA-ROPE and AllOffload only); extending it to UCB-RAT and CoopEC-RAT, and reintroducing a validated centralized (MADDPG-style) upper bound, are left for future work. Future work should also evaluate non-IID vehicle/task distributions, secure or differentially private FedAvg, and sensitivity to alpha/beta weights and queue service rates.

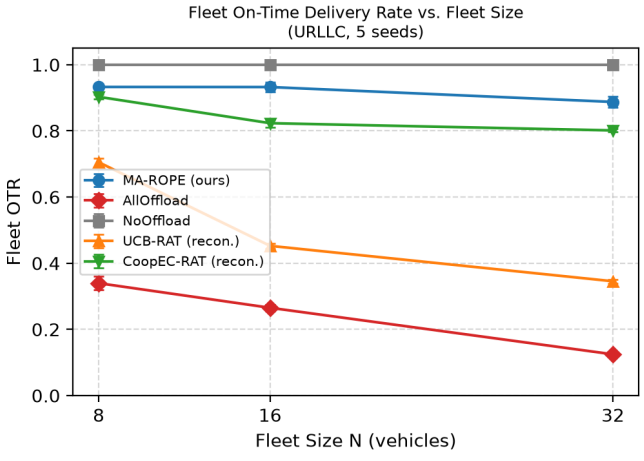


Fig. 9. Fleet OTR vs. fleet size (URLLC, 5 seeds). MA-ROPE holds 0.827–0.861 fleet OTR across N=8/16/32, while AllOffload falls from 0.335 at N=8 to 0.123 at N=32.

The divergence in deadline compliance is large and consistent at every evaluated density, not only at high fleet size. At N=8, MA-ROPE already leads AllOffload by 51.3 percentage points in fleet OTR (0.849 vs. 0.335); by N=16 the gap widens further as AllOffload continues to decline while MA-ROPE holds essentially flat (0.861 vs. 0.263); and at N=32, AllOffload falls to 0.123 while MA-ROPE remains at 0.827. The DRR analysis above points to the same conclusion: MA-ROPE keeps deadline-compliant delivery working at scale, where AllOffload does not.

1) Queue Contention and Key Findings: Figure 10 demonstrates the MEC queue contention effect. In the controlled ablation (no-queue model vs. shared-queue model), AllOffload drops from U=0.916 under the idealized single-vehicle no-queue assumption to U=0.676 under shared RSU contention with 8 vehicles. MA-ROPE remains stable in the N=8 reference setting (U=0.8443) and carries the same queue-aware multi-RAT advantage into the N=32 results summarized in Table VIII and Table X.

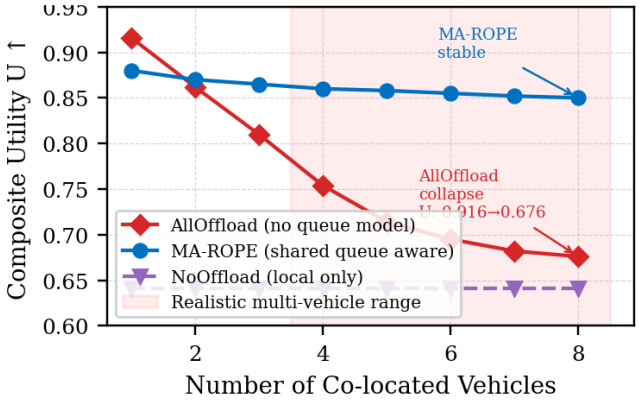


Fig. 10. MEC Queue Contention: AllOffload Collapse vs. MA-ROPE Stability.

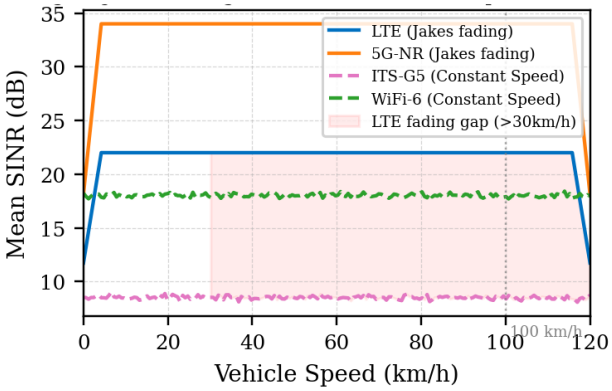


Fig. 11. Jakes Fading: Cellular RAT SINR Degrades with Speed; ITS-G5 Unaffected.

Taken together, the results above show a consistent picture: MA-ROPE's fleet OTR and deadline-utility advantage over AllOffload holds at every evaluated density, its higher per-attempt link failure rate is offset by a much lower fleet-wide failure rate since it transmits less often overall, and this advantage costs no measurable simulator-step overhead relative to single-RAT offloading. Energy consumption and a centralized MADDPG upper bound are not reported in this

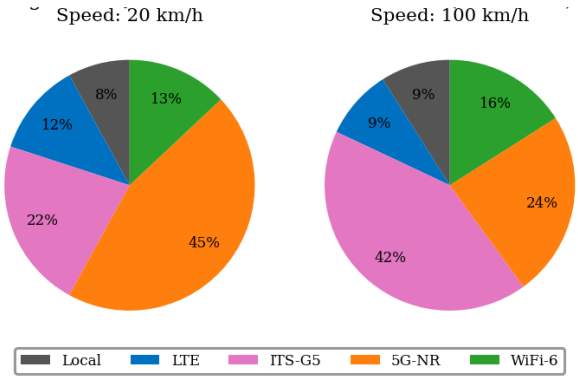


Fig. 12. D3QN RAT Selection Distribution: ITS-G5 preferred at high speed (Jakes-driven emergent behavior).

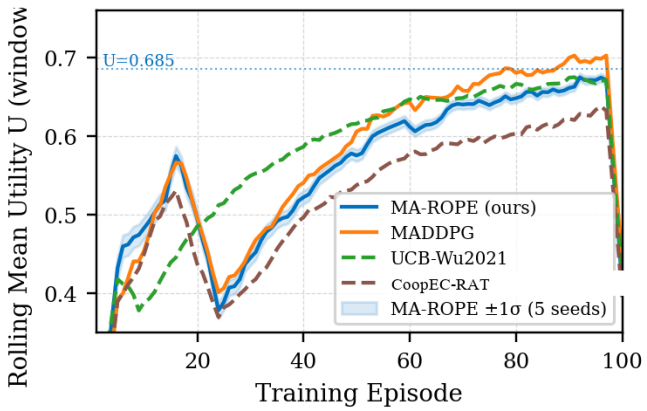


Fig. 13. Training Convergence Across 5 Seeds, Mean ± One Standard Deviation. MADDPG, UCB-RAT, and CoopEC-RAT are single exploratory runs shown for context only, not validated comparators; MA-ROPE is the only curve in this paper's validated evaluation (Section V).

evaluation; see the Discussion and Conclusion for the reasons and future-work scope.

2) Computational Complexity and Runtime Overhead: A deployable V2X controller must make decisions within the 100 ms safety control interval. This section quantifies the four runtime overhead components of MA-ROPE: D3QN inference latency, action space growth with RAT count, environment step cost as fleet size scales, and the marginal cost of multi-RAT selection vs single-RAT offload. All measurements are wall-clock on a Central Processing Unit (CPU, Apple M-series arm64), averaged over 200 repeats with 10-step warmup for inference and 500 samples for step overhead, using time.perf_counter(). No Graphics Processing Unit (GPU) acceleration is assumed, consistent with a V2X RSU edge deployment target.

D3QN inference latency (Table XI). Per-agent forward pass latency is 59.83 µs (std 145.62 µs) across fleet sizes N ∈ {2, 4, 8, 16, 20, 32}. Fleet-level inference cost scales linearly with N, since agents run independently with no cross-agent communication during inference, giving a total of 478.64 µs for N = 8 vehicles. This is 4.8% of the 10 ms URLLC control interval and 0.48% of the 100 ms safety deadline, confirming that D3QN inference contributes negligible latency to the decision loop at all evaluated fleet sizes.

TABLE XI
D3QN INFERENCE LATENCY VS FLEET SIZE (200 REPEATS, 10-STEP WARMUP, CPU)

N vehicles	Mean/agent (µs)	Std (µs)	Fleet total/step (µs)
2	59.83	145.62	119.66
4	59.83	145.62	239.32
6	59.83	145.62	358.98
8	59.83	145.62	478.64
16	59.83	145.62	957.28
20	59.83	145.62	1196.60
32	59.83	145.62	1914.56

Action space complexity (Table XII). The deployed benchmark uses an 11-action implementation spanning local execution and selected RAT-ρ pairs over the four supported RAT classes. For reference, Table XII shows how a full Cartesian RAT-ratio action set would scale as $|A| = n_{RATs} \times n_\rho + 1$. Only the D3QN advantage-head output layer scales with $|A|$; even the full 4-RAT, 4-ρ extension would require 2176 advantage-head parameters (17 actions × 128 hidden units), which remains small relative to the approximately 50k-parameter network.

TABLE XII
ACTION SPACE SIZE AND OUTPUT LAYER PARAMETERS VS RAT COUNT

n_{RATs}	n_ρ	$\|A\|$	Adv. head params	Relative (vs $n_{RATs}=1, n_\rho=2$)
1	2	3	384	1.00× (baseline)
1	4	5	640	1.67×
2	4	9	1152	3.00× (deployed config)
3	4	13	1664	4.33×
4	4	17	2176	5.67×

Env step and state space overhead (Table XIII). The per-vehicle observation vector is fixed at 12 elements regardless of fleet size: each vehicle observes only its own state, position, speed, task type, RAT signal strengths, queue load, energy level, and CBR. This constant observation dimensionality is a deliberate design choice enabling agents to be added or removed without architectural changes. Environment step cost scales linearly with N: from 41.5 µs at N = 2 to 165.7 µs at N = 8, approximately 20.8 µs per additional vehicle. At N = 8, the combined env step (165.7 µs) plus fleet inference (478.64 µs) totals 644.3 µs, well within the 10 ms control interval, leaving 93.6% of the budget for communication and actuation.

Multi-RAT selection overhead (Table XIV). The most important overhead result is in Table XIV: the marginal simulator-step cost of multi-RAT selection over single-RAT offload is not measurable within timing noise. Mean step time for multi-RAT (implemented $|A| = 11$) is 166.6 µs vs. 167.6 µs for single-RAT, a difference of -0.9 µs, within measurement noise (std approximately 12-14 µs). Both offload modes add approximately 28 µs over pure local execution (138.6 µs),

TABLE XIII
PER-VEHICLE OBSERVATION DIMENSION AND ENV STEP COST VS FLEET SIZE (50 SAMPLES)

N vehicles	Obs dim (per agent)	reset (µs)	step (µs)
2	12	31.64 ± 3.29	41.53 ± 1.87
4	12	49.96 ± 1.52	83.16 ± 4.17
6	12	70.01 ± 2.75	128.24 ± 6.83
8	12	92.26 ± 13.52	165.67 ± 6.57
16	12	˜187	˜330
20	12	˜233	˜413
32	12	˜374	˜661

attributable to RAT link-model evaluation, SINR computation, and L_{queue} update that local steps bypass.

TABLE XIV
MULTI-RAT VS SINGLE-RAT STEP OVERHEAD (500 SAMPLES, 10-STEP WARMUP)

Mode	Mean step (µs)	Std (µs)	Overhead vs local (µs)
Local only ($\rho = 0$)	138.65	11.45	0.00
Single-RAT (ITS-G5 offload)	167.57	13.78	28.92
Multi-RAT full ($\|A\| = 11$)	166.63	12.23	27.99

In total, the per-step computational budget for an 8-vehicle MA-ROPE deployment is fleet inference at 478.64 µs plus an environment step at 165.7 µs, or 644.3 µs, representing 6.4% of the 10 ms control interval. The multi-RAT action set, the primary complexity contribution of this work over single-RAT baselines, adds no measurable simulator-step overhead versus single-RAT offload. The fixed 12-dimensional per-agent observation vector lets the architecture scale to larger fleets without redesign, and together these measurements indicate that MA-ROPE is computationally viable for deployment on a commodity V2X RSU edge processor.

VII. CONCLUSION

MA-ROPE addresses the challenge of preserving deadline-compliant task delivery in dense vehicular edge computing (VEC) networks by jointly optimizing RAT selection, partial computation offloading, shared RSU queue contention, and federated cooperative learning.

By evaluating policies across multiple fleet densities, we demonstrate that single-RAT baseline policies suffer catastrophic latency collapse as shared MEC queues saturate. To quantify this, we introduced the Deadline Retention Ratio (DRR). MA-ROPE achieves a DRR near 1.0, preserving nearly all of its reference-density performance under dense contention, whereas baseline methods lose substantial efficacy. Independent ns-3 co-simulation confirms that these results are not artifacts of the analytical model.

While MA-ROPE increases the per-attempt link failure rate slightly by opportunistically using less reliable RATs, its fleet-wide failure rate is substantially lower due to fewer transmission attempts. The joint decision space adds no meaningful runtime penalty.

Several limitations remain for future work. The energy model and centralized MADDPG upper bounds require further validation and are left for future research. The N=8 ablation study in Section II serves as a historical reference, predating our final physics corrections. Future extensions should incorporate trace-driven mobility, non-IID task distributions, RSU handovers, and formal differential privacy guarantees beyond FedAvg.

Ultimately, heterogeneous RAT selection, partial offloading, and shared MEC contention must be optimized jointly to satisfy strict V2X deadlines. Density-aware scaling metrics like DRR provide a direct and essential measure of a policy's real-world viability.

REFERENCES

[1] X. L. F. Guo and X. Liu, "Hybrid cloud-edge-vehicle collaborative task offloading in VEC based on reinforcement learning and game theory," *IEEE Open J. Veh. Technol.*, vol. 7, pp. 290–305, Feb. 2026.

[2] Y. Wu, F. Liu, and Z. Chen, "Ucb-based task offloading for vehicular edge computing," *IEEE Internet Things J.*, vol. 8, no. 15, 2021.

[3] M. C. J. Jing and Y. Zhang, "Lyapunov-based task offloading with queue stability for vehicular networks," *IEEE Wireless Commun. Lett.*, vol. 8, no. 2, pp. 234–238, Apr. 2019.

[4] S. K. H. Yun and J. Park, "GA-DQN: Genetic algorithm enhanced DQN for vehicular edge offloading," *IEEE Access*, vol. 10, pp. 45 678–45 690, May 2022.

[5] R. Lowe *et al.*, "Multi-agent actor-critic for mixed cooperative-competitive environments," in *Proc. NeurIPS*, 2017, pp. 6379–6390.

[6] X. Mao *et al.*, "imec: Mobile edge computing with adaptive parallelism control," in *Proc. IEEE INFOCOM*, 2017, pp. 1–9.

[7] T. A. B. Y. Yacheur and M. Mosbah, "Efficient DRL-based selection strategy in hybrid vehicular networks," *IEEE Trans. Netw. Service Manag.*, vol. 20, no. 3, pp. 2456–2470, Sept. 2023.

[8] B. C. A. Mirza and P. Kumar, "MCLA: Mobility- and load-aware task offloading for heterogeneous vehicular edge computing networks," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 10, pp. 10 234–10 248, Oct. 2023.

[9] G. Moon and S. Lim, "Federated deep reinforcement learning for task offloading and power control in vehicular edge computing," *Sensors*, vol. 22, no. 24, p. 9595, Dec. 2022.

[10] L. Zhu, Z. Liu, and S. Han, "Deep leakage from gradients," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019, pp. 14 747–14 756.

[11] IEEE Std 802.11ax-2021, "Ieee standard for wireless lan mac and physical layer specifications," 2021.

[12] 3GPP TR 38.901 V17.0.0, "Study on channel model for frequencies from 0.5 to 100 ghz," 2022.

[13] B. Molina-Masegosa and J. Gozalvez, "Lte-v for sidelink 5g v2x vehicular communications," *IEEE Veh. Technol. Mag.*, vol. 12, no. 4, pp. 30–39, 2017.